Practical 1 -: To perform Version Control using GIT

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency. Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

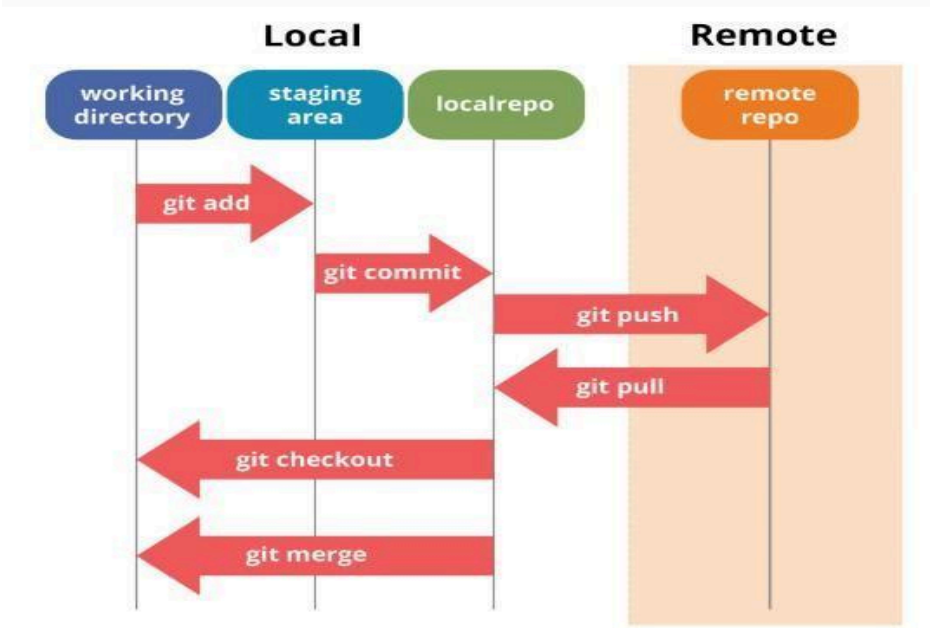
Some of the basic operations in Git are:

1. Initialize
2. Add
3. Commit
4. Pull
5. Push

Some advanced Git operations are:

1. Branching
2. Merging
3. Rebasing

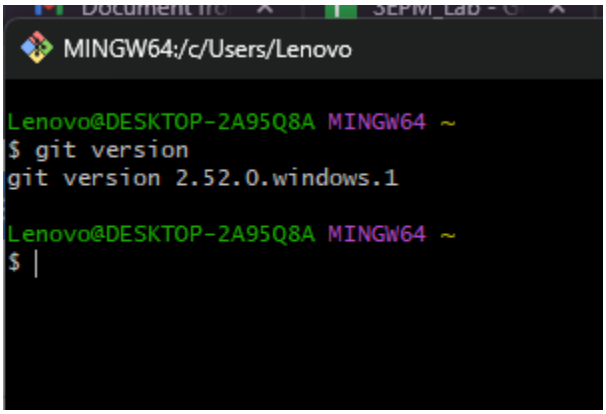
The following diagram depict the all supported operations in GIT



Installation of GIT

1) In windows, download GIT from https://git-scm.com/ and perform the straightforward installation.

The output of GIT Bash in windows shell is as below:



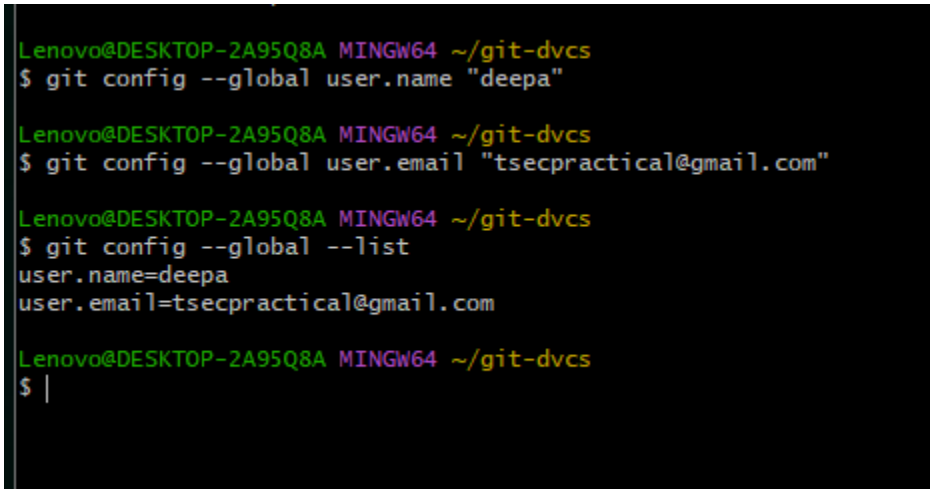
To perform version control, let us create a directory dvcs (Distributed version control system) and change directory to dvcs.

$ mkdir git-dvcs

$ cd git-dvcs/

Now check the user information using

```
$ git config --global
As there are no users defined, let us define it using following two commands
$ git config --global user.name "deepa"
$ git config --global user.email "tsecpractical@gmail.com"

Now, check the list of users
$ git config --global --list
user.name=bhushan
user.email=bhushan.jadhav1@gmail.com
```




```
Let us create a repository for version control named ”git-demo-project”
$ mkdir git-demo-project
$ cd git-demo-project/
Now, initialize the repository using following command
$ git init
```

The output of above command shown below which adds .git hidden directory in current repository.

```
Lenovo@DESKTOP-2A95Q8A MINGW64 ~/git-dvcs
$ git init
Initialized empty Git repository in C:/Users/Lenovo/git-dvcs/.git/

Lenovo@DESKTOP-2A95Q8A MINGW64 ~/git-dvcs (master)
$ |
```

Now, let us add some files inside our repository "`git-demo-project`"

To add files in the repository by create or copy some doc,html,image files inside current directory to see index and staging area. The add command is used along with dot (. Dot means current directory) for adding files in current repository i.e. making them in staging mode. They are untracked until we commit them.

```
$ git add .
Index and staging area
To check the status of repository, use
$ git status
```

Which will show you some untrack files, so untracks files can be tracked using commit command.

Now, let us commit the changes

```
$ git commit -m "First Commit" (#here -m for message)
```

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project
project (master)
$ git add .

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (m
aster)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (m
aster)
$ git commit -m "First Commit"
On branch master

Initial commit

nothing to commit (create/copy files and use "git add" to track)
```

Add index.html in our directory

```
$ git add .
$ git commit -am "express Commit" (#Here -a used for express commit)
$ nano index.html
```

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$  touch teststatus

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ git checkout -- teststatus
error: pathspec 'teststatus' did not match any file(s) known to git

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ git checkout -- index.html

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ |
```

```
$ touch teststatus
```

```
ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        teststatus

no changes added to commit (use "git add" and/or "git commit -a")
```

```
ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git checkout -- teststatus
error: pathspec 'teststatus' did not match any file(s) known to git

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git checkout -- index.html
```

Changes are Discarded by checkout

```
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
```

```
ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git add index.html

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        teststatus
```

$ git add index.html

$ git add teststatus

```
ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        teststatus


ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git add teststatus

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   index.html
        new file:   teststatus
```

$ git commit -am "Express commit"

```
MINGW64:/c/Users/Lenovo/git-dvcs/git-docs-T21-79/git-demo-project

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ git commit -am "Express commit"
[master c5a3d91] Express commit
 2 files changed, 11 insertions(+)
 create mode 100644 teststatus

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ git status
On branch master
nothing to commit, working tree clean

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$
```

Now let us see history of commits. The log command is used for seeing the commit history.

```
$ git log
```

```
Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ git log
commit c5a3d91e7868782463059bbe7f66a4356f404cf2 (HEAD -> master)
Author: aarya <aaryapatil2403@gmail.com>
Date:   Tue Jan 20 14:21:37 2026 +0530

    Express commit

commit 922742ff78d970d16c2b0d9e46eea83d2ac23463
Author: aarya <aaryapatil2403@gmail.com>
Date:   Tue Jan 20 14:09:42 2026 +0530

    express Commit

Lenovo@DESKTOP-ADUA2JI MINGW64 ~/git-dvcs/git-docs-T21-79/git-demo-project (master)
$ |
```

To see all the operation in oneline use the –-oneline option in log command

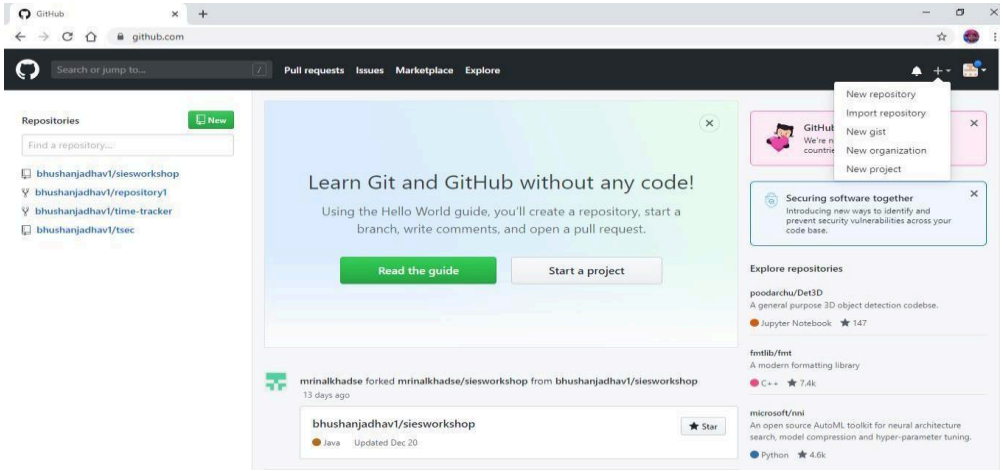
–-oneline option for particular file in log command

```
ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git log --oneline teststatus
d3a6a76 (HEAD -> master) Express commit

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git log --oneline
d3a6a76 (HEAD -> master) Express commit
97d0a76 express Commit
50148fb First Commit

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git log --oneline 97d0a76..d3a6a76
d3a6a76 (HEAD -> master) Express commit

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git log --oneline -n 2
d3a6a76 (HEAD -> master) Express commit
97d0a76 express Commit
```

Example 2: Performing Version control in GITHUB with Pull and Push commands

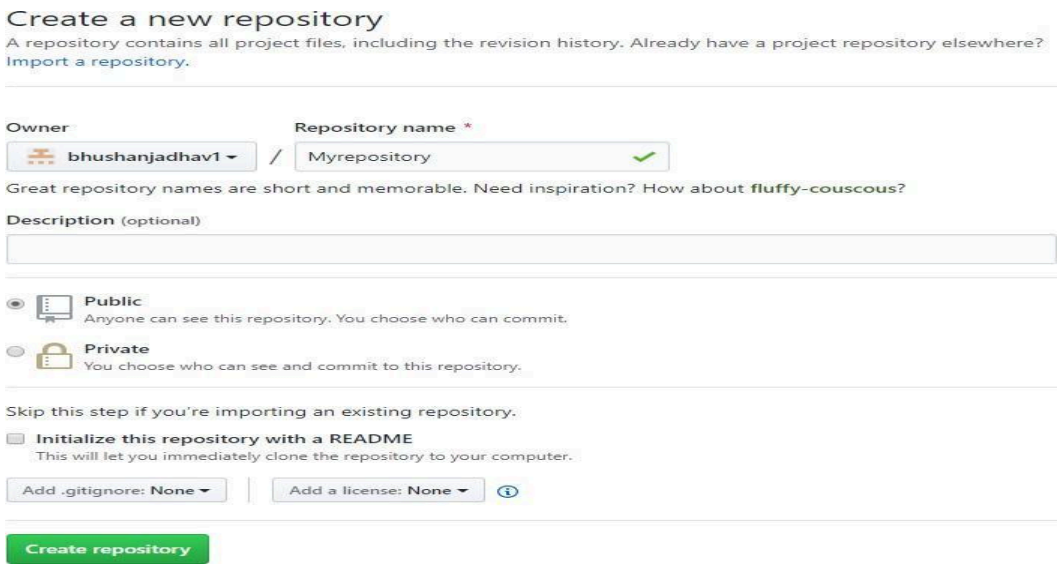
First open Github.com and create a new account.After verifying account through E-mail, create a Repository on github.com.

Open github.com create an account



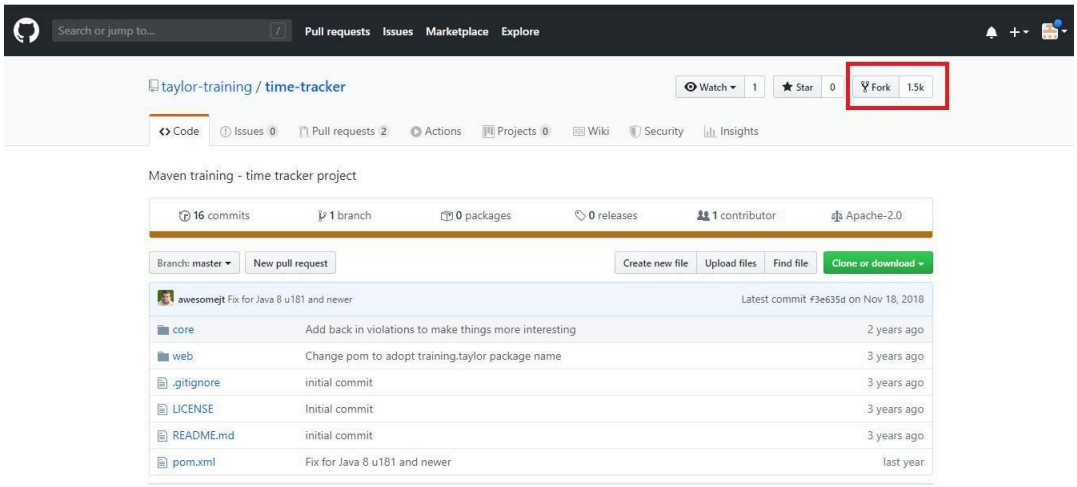
After login Select New repository from the menu.

Now Specify a Name to repository and select public option followed by create repository

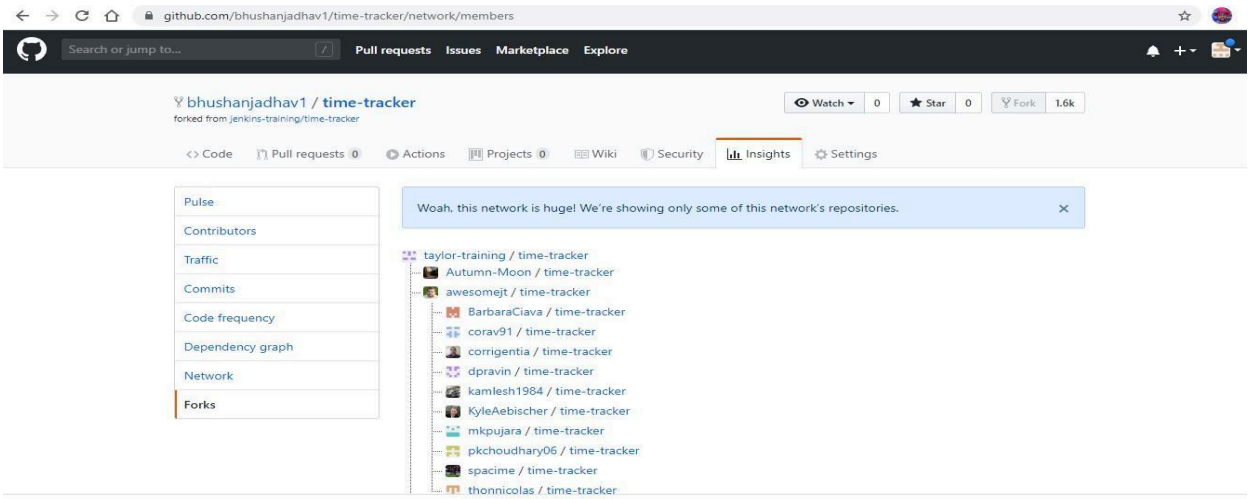


By default, we can create public repository in Github. So we can copy the entire public repository of any other users in to own account using “FORK” Operation. Now fork the repository (Sharing with other users who wants to contribute).

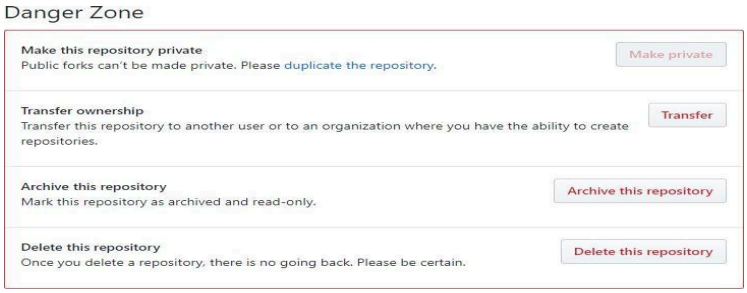
Login with another account Copy and Paste URL of repository then just click on fork to clone to others account. Suppose we want to fork public repository “timetracker”. So search for “timetracker” github repository on google and once its opened clicked on “Fork button” from the top of the github web page as shown below.

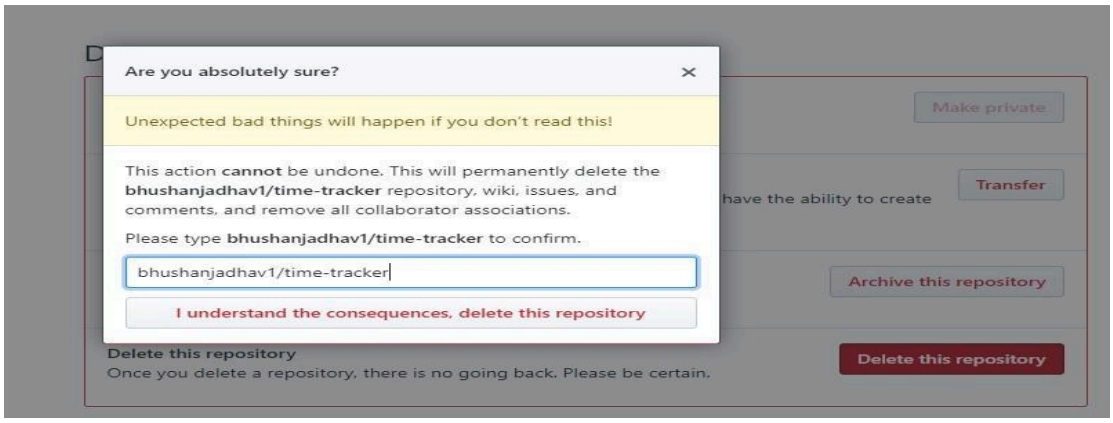


After fork it will be added in your local repository.

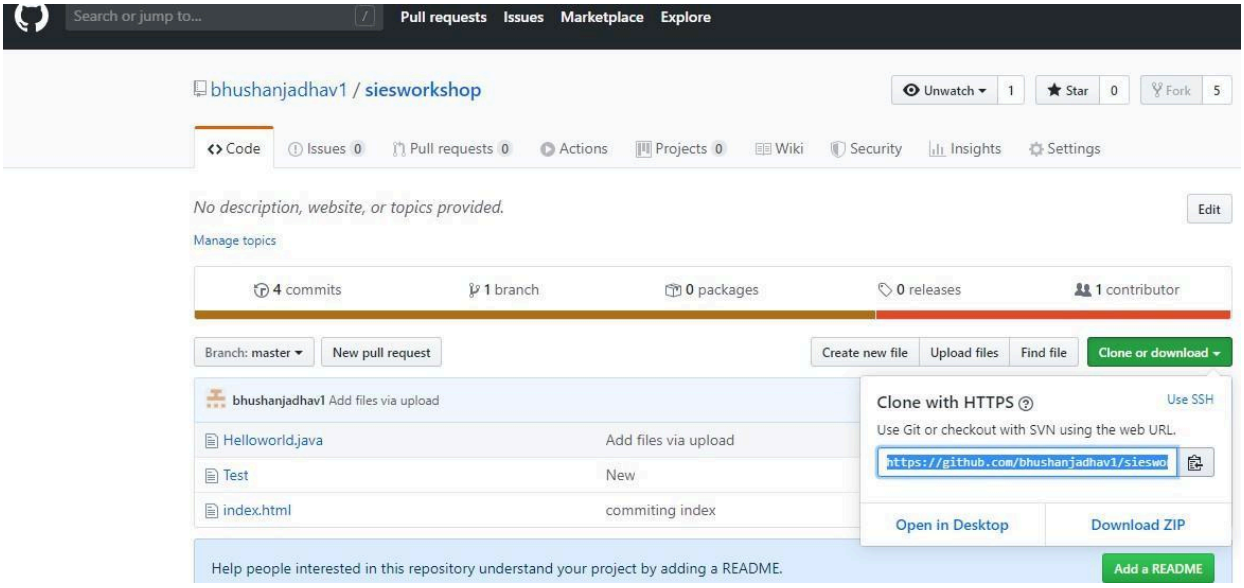


To delete the repository, open the desired repository you want to delete and go to the settings option. There you will see delete repository button to delete it.

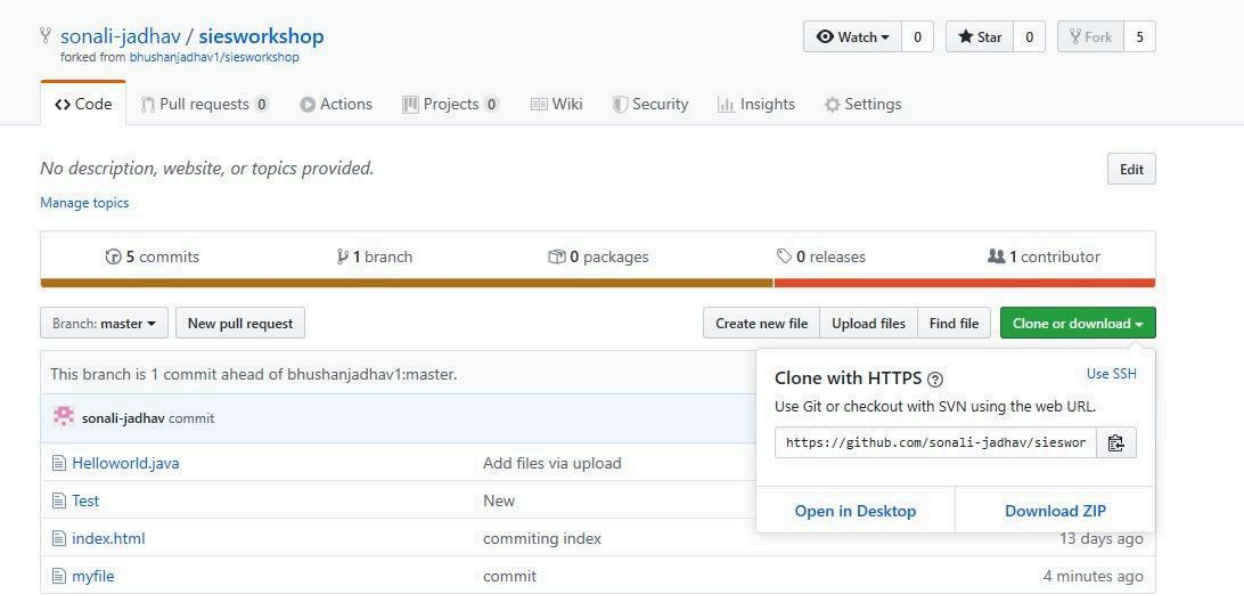




Now, if you want to download a repository in local machine, then git clone command is used followed by path to repository. In GitHub the path of repository can be known through clone or download button and it can be downloaded using git clone command as shown below.



```
ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ git clone https://github.com/bhushanjadhav1/siesworkshop
Cloning into 'siesworkshop'...
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (12/12), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 12 (delta 0), reused 9 (delta 0), pack-reused 0
Unpacking objects: 100% (12/12), done.

ADMIN@DESKTOP-J582V2L MINGW64 ~/Desktop/git-dvcs/git-demo-project (master)
$ ls
 Gitpracts.docx   index.html  'Installation and Configuration of GIT.docx'   siesworkshop/   teststatus
```



Pull and Push Processes

The pull command used to fetch the repository from github to local while push is used to commit files from local repository to Github.

Push Push changes to Web repository

Pull Pull changes to Local repository

The following commands are used for pull and push repositories

A) Push command

```
PS C:\Users\HP 15L\Desktop\T21-79> git init
Initialized empty Git repository in C:/Users/HP 15L/Desktop/T21-79/.git/
PS C:\Users\HP 15L\Desktop\T21-79>
```

$ git remote add origin https://github.com/bhushanjadhav1/siesworkshop.git

```
Initialized empty Git repository in C:/Users/HP 15L/Desktop/T21-79/.git/
PS C:\Users\HP 15L\Desktop\T21-79> git remote add origin https://github.com/tsecprac
tical-star/SEPM_PRAC.git
PS C:\Users\HP 15L\Desktop\T21-79>
```

$ git remote show origin

```
PS C:\Users\HP 15L\Desktop\T21-79> git remote show origin
* remote origin
  Fetch URL: https://github.com/tsecpractical-star/SEPM_PRAC.git
  Push  URL: https://github.com/tsecpractical-star/SEPM_PRAC.git
  HEAD branch: main
  Remote branch:
    main new (next fetch will store in remotes/origin)
PS C:\Users\HP 15L\Desktop\T21-79>
```

```
PS C:\Users\HP 15L\Desktop\T21-79> git commit -m "initial commit"
[master (root-commit) 0b77317] initial commit
 1 file changed, 10 insertions(+)
 create mode 100644 exp5.py
PS C:\Users\HP 15L\Desktop\T21-79> git push -u origin main
error: src refspec main does not match any
```

If you add remote again then will show you fatal error.

$ git remote add origin https://github.com/bhushanjadhav1/Myrepository.git

fatal: remote origin already exists.

So, to delete origin rm origin command is used

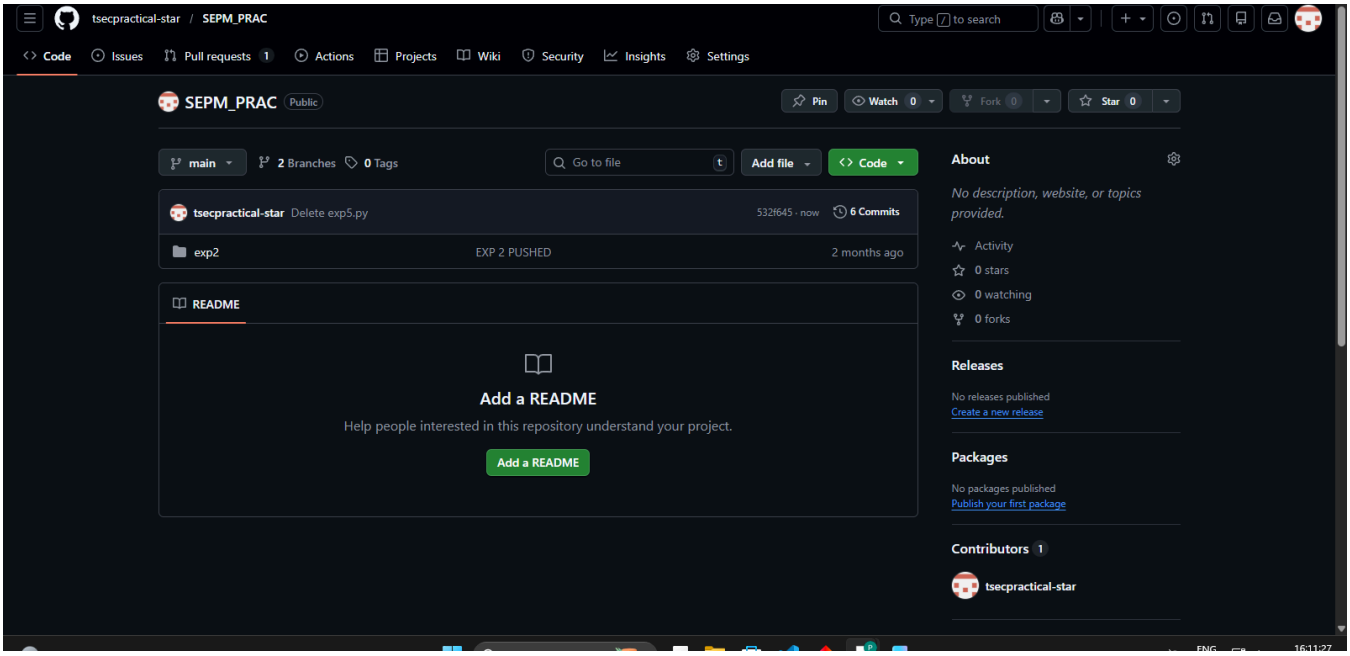
$ git remote rm origin

Now, to push the local repository to remote github following command is used

$ git push -u origin master

```
PS C:\Users\HP 15L\Desktop\T21-79> git branch -m Main
PS C:\Users\HP 15L\Desktop\T21-79> git push -u origin Main
info: please complete authentication in your browser...
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 24 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 657 bytes | 657.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Main' on GitHub by visiting:
remote:      https://github.com/tsecpractical-star/SEPM_PRAC/pull/new/Main
remote:
To https://github.com/tsecpractical-star/SEPM_PRAC.git
 * [new branch]      Main -> Main
branch 'Main' set up to track 'origin/Main'.
PS C:\Users\HP 15L\Desktop\T21-79>
```

Now you can check the github for updated contents.



B) Pull Changes

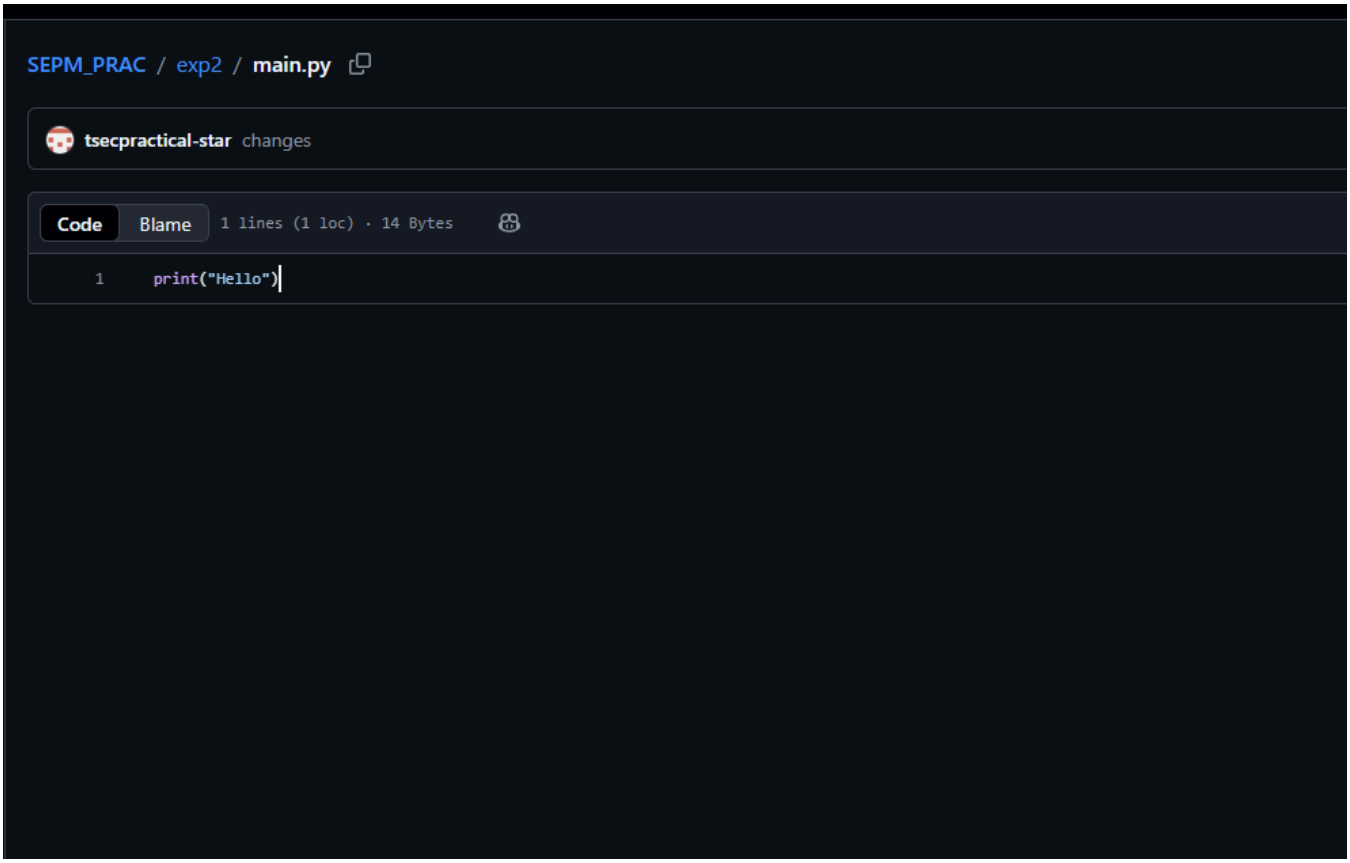
Pull command is used to download the remote updated repository into local one. The command for download is

```
$ git pull
```

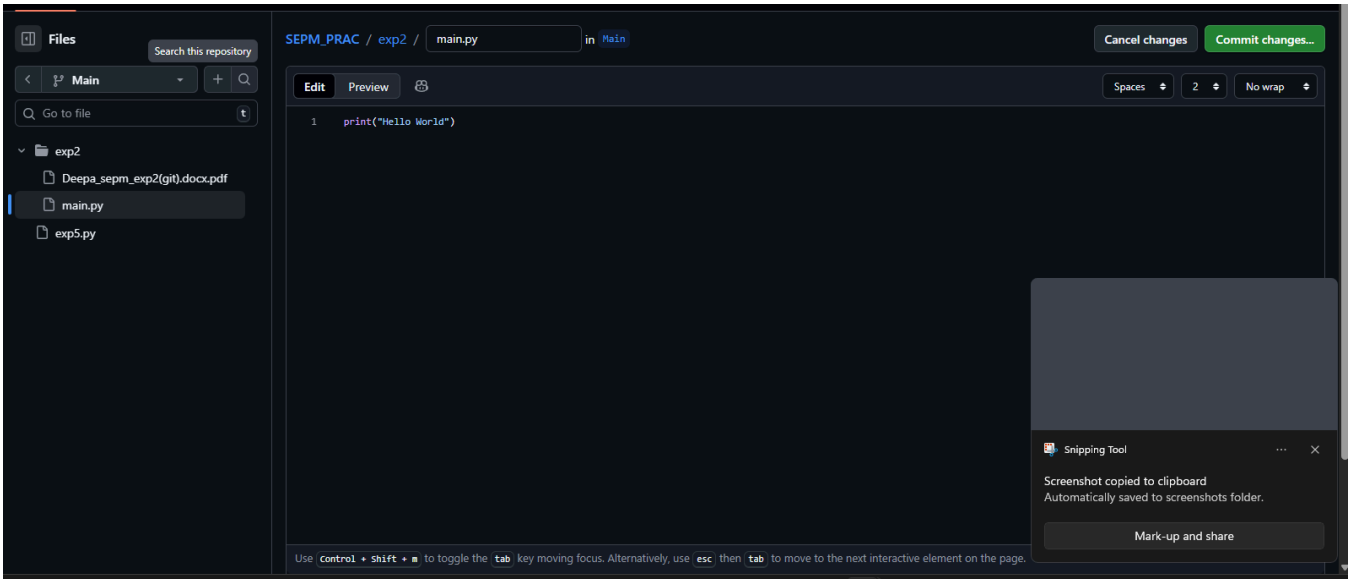
```
PS C:\Users\HP 15L\Desktop\T21-79> git pull
remote: Enumerating objects: 2, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 1.66 KiB | 113.00 KiB/s, done.
From https://github.com/tsecpractical-star/SEPM_PRAC
 * [new branch]      main       -> origin/main
Already up to date.
PS C:\Users\HP 15L\Desktop\T21-79>
```

C) Fetch

Suppose you have a file in github and you have changes that.



ORIGINAL FILE



Changed File

Now we use fetch command to fetch the changes, which will show you both the files like original and changed in local repository.

$ git fetch

```
PS C:\Users\HP 15L\Desktop\T21-79> git fetch
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 2.61 KiB | 178.00 KiB/s, done.
From https://github.com/tsecpractical-star/SEPM_PRAC
   011b9ae..09bf1dd  Main       -> origin/Main
 * [new branch]      main       -> origin/main
PS C:\Users\HP 15L\Desktop\T21-79>
```

Here fetch will not show you like updated changes file as like push. So use merge command to merge the changes so use following command for merge.

$ git merge origin/master

```
PS C:\Users\HP 15L\Desktop\T21-79> git merge origin/Main
>>
Merge made by the 'ort' strategy.
 exp5.py | 10 ----------
 1 file changed, 10 deletions(-)
 delete mode 100644 exp5.py
PS C:\Users\HP 15L\Desktop\T21-79>
Main  0↓ 2↑  ⊗ 0 ⚠ 0  0                    Ln 1, Col 15   Spaces: 4   UTF-8
```

Online Resource: https://dzone.com/articles/top-20-git-commands-with-examples